

调试与运维

MoonPalace - Moonshot AI 月之暗面 Kimi API 调试工具

📄 复制页面



MoonPalace（月宫）是由 Moonshot AI 月之暗面提供的 API 调试工具。它具备以下特点：

- 全平台支持：
 - ☒ Mac
 - ☒ Windows
 - ☒ Linux
- 简单易用，启动后将 `base_url` 替换为 `http://localhost:9988` 即可开始调试；
- 捕获完整请求，包括网络错误时的“事故现场”；
- 通过 `request_id`、`chatcmpl_id` 快速检索、查看请求信息；
- 一键导出 BadCase 结构化上报数据，帮助 Kimi 完善模型能力；

我们推荐在代码编写和调试阶段使用 MoonPalace 作为你的 API “供应商”，以便能快速发现和定位关于 API 调用和代码编写过程中的各种问题，对于 Kimi 大模型各种不符合预期的输出，你也可以通过 MoonPalace 导出请求详情并提交给 Moonshot AI 以改进 Kimi 大模型。

安装方式

使用 `go` 命令安装

如果你已经安装了 `go` 工具链，你可以执行以下命令来安装 MoonPalace：

KIMI 开放平台



上述命令会在你的 `$GOPATH/bin/` 目录安装编译后的二进制文件，运行 `moonpalace` 命令来检查是否成功安装：

```
$ moonpalace
MoonPalace is a command-line tool for debugging the Moonshot AI HTTP API.

Usage:
  moonpalace [command]

Available Commands:
  cleanup      Cleanup Moonshot AI requests.
  completion   Generate the autocompletion script for the specified shell
  export       export a Moonshot AI request.
  help         Help about any command
  inspect      Inspect the specific content of a Moonshot AI request.
  list         Query Moonshot AI requests based on conditions.
  start        Start the MoonPalace proxy server.

Flags:
  -h, --help      help for moonpalace
  -v, --version    version for moonpalace

Use "moonpalace [command] --help" for more information about a command.
```

如果你仍然无法检索到 `moonpalace` 二进制文件，请尝试将 `$GOPATH/bin/` 目录添加到你的 `$PATH` 环境变量中。

从 Releases 页面下载

你可以从 [Releases](#) 页面下载编译好的二进制（可执行）文件：

- moonpalace-linux
- moonpalace-macos-amd64 ⇒ 对应 Intel 版本的 Mac

- moonpalace-windows.exe

KIMI 开放平台



请根据自己的平台下载对应的二进制（可执行）文件，并将二进制（可执行）文件放置在已被包含在环境变量 `$PATH` 中的目录中，将其更名为 `moonpalace`，最后为其赋予可执行权限。

使用方式

启动服务

使用以下命令启动 MoonPalace 代理服务器：

```
$ moonpalace start --port <PORT>
```

MoonPalace 会在本地启动一个 HTTP 服务器，`--port` 参数指定 MoonPalace 监听的本地端口，默认值为 `9988`。当 MoonPalace 启动成功时，会输出：

```
[MoonPalace] 2024/07/29 17:00:29 MoonPalace Starts => change base_url to "http://127.0.0.1:9988/v1"
```

按照要求，我们将 `base_url` 替换为显示的地址即可，如果你使用默认的端口，那么请设置 `base_url=http://127.0.0.1:9988/v1`，如果你使用了自定义的端口，请将 `base_url` 替换为显示的地址。

额外的，如果你想在调试时始终使用一个调试的 `api_key`，你可以在启动 MoonPalace 时使用 `-key` 参数为 MoonPalace 设定一个默认的 `api_key`，这样你就可以不用在请求时手动设置 `api_key`，MoonPalace 会帮你在请求 Kimi API 时添加你通过 `--key` 设定的 `api_key`。

如果你正确设置了 `base_url`，并成功调用 Kimi API，MoonPalace 会输出如下的信息：

```
[MoonPalace] 2024/07/29 17:00:29 MoonPalace Starts => change base_url to "http://1
KIMI 开放平台 [MoonPalace] 2024/07/29 21:30:53 POST /v1/chat/completions 200 OK
[MoonPalace] 2024/07/29 21:30:53 - Request Headers:
[MoonPalace] 2024/07/29 21:30:53 - Content-Type: application/json
[MoonPalace] 2024/07/29 21:30:53 - Response Headers:
[MoonPalace] 2024/07/29 21:30:53 - Content-Type: application/json
[MoonPalace] 2024/07/29 21:30:53 - Msh-Request-Id: c34f3421-4dae-11ef-b237-962
[MoonPalace] 2024/07/29 21:30:53 - Server-Timing: 7134
[MoonPalace] 2024/07/29 21:30:53 - Msh-Uid: cn0psmmcp7fcInphkcpG
[MoonPalace] 2024/07/29 21:30:53 - Msh-Gid: enterprise-tier-5
[MoonPalace] 2024/07/29 21:30:53 - Response:
[MoonPalace] 2024/07/29 21:30:53 - id: cmpl-12be8428ebe74a9e846
[MoonPalace] 2024/07/29 21:30:53 - prompt_tokens: 1449
[MoonPalace] 2024/07/29 21:30:53 - completion_tokens: 158
[MoonPalace] 2024/07/29 21:30:53 - total_tokens: 1607
[MoonPalace] 2024/07/29 21:30:53 New Row Inserted: last_insert_id=15
```

MoonPalace 会以日志的形式将请求的细节在命令行中输出（假如你想将日志的内容持久化存储，你可以将 `stderr` 重定向到文件中）。

注：在日志中，Response Headers 中的 `Msh-Request-Id` 字段的值对应下文中检索请求、导出请求中的 `--requestid` 参数的值，Response 中的 `id` 对应 `--chatcmpl` 参数的值，`last_insert_id` 对应 `--id` 参数的值。

```
[MoonPalace] 2024/08/05 19:06:19 it seems that your max_tokens value is too small
```

如果当前使用的是非流式输出模式（`stream=False`），MoonPalace 会给出建议的 `max_tokens` 值。

启用重复内容输出检测

MoonPalace 提供了对 Kimi 大模型重复内容输出的检测功能。重复内容输出指的是：**Kimi 大模型会重复不断地输出某一特定字词、句子以及空白字符，并且在达到 `max_tokens` 限制前不会停下来。**在使用 `moonshot-v1-128k` 等费用较高的模型时，这种重复输出会导致额外的 Tokens

小·

KIMI 开放平台



```
$ moonpalace start --port <PORT> --detect-repeat --repeat-threshold 0.3 --repeat-r
```

启用 `--detect-repeat` 选项后，MoonPalace 会在检测到 Kimi 大模型的重复内容输出行为时，中断 Kimi 大模型输出，并在日志中输出：

```
[MoonPalace] 2024/08/05 18:20:37 it appears that there is an issue with content
```

注：启用 `--detect-repeat` 后，仅在流式输出 (`stream=True`) 的场合，MoonPalace 会中断 Kimi 大模型的输出，非流式输出场合不适用。

你可以使用 `--repeat-threshold` / `--repeat-min-length` 参数来调整 MoonPalace 的阻断行为：

- `--repeat-threshold` 参数用于设置 MoonPalace 对重复内容的容忍度，越高的 threshold 表示容忍度越低，重复内容将更快被阻断，`0 <= threshold <= 1`。
- `--repeat-min-length` 参数用于设置 MoonPalace 检测重复内容输出的起始字符数量，例如：`--repeat-min-length=100` 表示当输出的 utf-8 字符数超过 100 时开启重复检测，输出字符数小于 100 时不开启重复内容输出检测

启用强制流式输出

MoonPalace 提供了 `--force-stream` 的选项来强制让所有的 `/v1/chat/completions` 请求都使用流式输出模式：

```
$ moonpalace start --port <PORT> --force-stream
```

MoonPalace 会将请求参数中的 `stream` 字段设置为 `True`，并在获得响应时，自动根据调用方是否设置了 `stream` 来决定响应的格式：

特殊处理。

KIMI 开放平台

如果调用方没有设置 `stream` 的值，或设置了 `stream=False`，MoonPalace 会在接收完所有流式数据块后，将数据块拼接成完整的 completion 结构返回给调用方。

对于调用方（开发者）而言，启用 `--force-stream` 选项不会你获得的 Kimi API 响应内容，你仍然可以使用原先的代码逻辑来调试和运行你的程序，换句话说：**开启 `--force-stream` 选项不会改变和破坏任何事物**，你可以放心地开启这个选项。

为什么要提供这样的选项？

我们初步推测常见的网络连接错误、超时等问题（Connection Error/Timeout）出现的原因是，在使用非流式模式进行请求的场合（`stream=False`），由于各中间层的网关或代理服务器对 `read_header_timeout` 或 `read_timeout` 进行了设置，导致当 Kimi API 服务端还在组装响应时，中间层的网关或代理服务器就断开了连接（由于没有收到响应，甚至是响应的 Header），产生 Connection Error/Timeout。我们尝试给 MoonPalace 添加了 `--force-stream` 参数，通过 `moonpalace start --force-stream` 启动时，MoonPalace 会将所有非流式请求（`stream=False` 或未设置 `stream`）转换为流式请求，并在接收完所有数据块后，组装成完整的 completion 响应结构返回给调用方。对于调用方而言，仍然可以使用原先的方式使用非流式 API，但经过 MoonPalace 的转换，能一定程度上减少 Connection Error/Timeout 的情况，因为此时 MoonPalace 已经与 Kimi API 服务端建立连接，并开始接收流式数据块。

检索请求

在 MoonPalace 启动后，所有经过 MoonPalace 中转的请求都将被记录在一个 sqlite 数据库中，数据库所在的位置是 `$HOME/.moonpalace/moonpalace.sqlite`。你可以直接连接 MoonPalace 数据库以查询请求的具体内容，也可以通过 MoonPalace 命令行工具来查询请求：

KIMI 开放平台		chatcmpl	request_id	
id	status			
15	200	cmpl-12be8428ebe74a9e8466a37bee7a9b11	c34f3421-4dae-11ef-b23	
14	200	cmpl-1bf43a688a2b48eda80042583ff6fe7f	c13280e0-4dae-11ef-9c0	
13	200	chatcmpl-2e1aa823e2c94ebdad66450a0e6df088	c07c118e-4dae-11ef-b42	
12	200	cmpl-e7f984b5f80149c3adae46096a6f15c2	50d5686c-4d98-11ef-ba6	
11	200	chatcmpl-08f7d482b8434a869b001821cf0ee0d9	4c20f0a4-4d98-11ef-999	
10	200	chatcmpl-6f3cf14db8e044c6bfd19689f6f66eb4	49f30295-4d98-11ef-95c	
9	200	cmpl-2a70a8c9c40e4bcc9564a5296a520431	7bd58976-4d8a-11ef-999	
8	200	chatcmpl-59887f868fc247a9a8da13cfbb15d04f	ceb375ea-4d7d-11ef-bd6	
7	200	cmpl-36e5e21b1f544a80bf9ce3f8fc1fce57	cd7f48d6-4d7d-11ef-999	
6	200	cmpl-737d27673327465fb4827e3797abb1b3	cc6613ac-4d7d-11ef-95c	

使用 `list` 命令将查询最近产生的请求内容，默认展示的字段是便于检索的

`id` / `chatcmpl` / `request_id` 以及用于查看请求状态的

`status` / `server_timing` / `requested_at` 信息。如果你想查看某个具体的请求，你可以使用

`inspect` 命令来检索对应的请求：

```
$ moonpalace inspect --id 13
moonpalace inspect --chatcmpl chatcmpl-2e1aa823e2c94ebdad66450a0e6df088
$ moonpalace inspect --requestid c07c118e-4dae-11ef-b423-62db244b9277
```

```
+-----+
| metadata |
+-----+
| {
|   "chatcmpl": "chatcmpl-2e1aa823e2c94ebdad66450a0e6df088",
|   "content_type": "application/json",
|   "group_id": "enterprise-tier-5",
|   "moonpalace_id": "13",
|   "request_id": "c07c118e-4dae-11ef-b423-62db244b9277",
|   "requested_at": "2024-07-29 21:30:43",
|   "server_timing": "1033",
|   "status": "200 OK",
|   "user_id": "cn0psmmcp7fcInphkcpG"
| }
+-----+
```

在默认情况下，`inspect` 命令不会打印出请求和响应的 body 信息，如果你想打印出 body，你可以使用如下的命令：

```
$ moonpalace inspect --chatcmpl chatcmpl-2e1aa823e2c94ebdad66450a0e6df088 --print
```

由于 body 信息过于冗长，这里不再完整展示 body 详细内容

```
+-----+
| request_body | response_body |
+-----+
| ...         | ...         |
+-----+
```

导出请求

当你认为某个请求不符合预期，或是想向 Moonshot AI 报告某个请求时（无论是 Good Case 还是 Bad Case，我们都欢迎），你可以使用 `export` 命令导出特定的请求：


```
$ moonpalace export \
```

KIMI 开放平台



```
--chatcmpl chatcmpl-2e1aa823e2c94ebdad66450a0e6df088 \  
--requestid c07c118e-4dae-11ef-b423-62db244b9277 \  
--good/--bad \  
--tag "code" --tag "python" \  
--directory $HOME/Downloads/
```

其中，`id` / `chatcmpl` / `requestid` 用法与 `inspect` 命令相同，用于检索一个特定的请求，`-good` / `--bad` 用于标记当前请求是 Good Case 或是 Bad Case，`--tag` 用于为当前请求打上对应的标签，例如在上述例子中，我们假设当前请求内容与编程语言 Python 相关，因此为其添加两个 `tag`，分别是 `code` 和 `python`，`--directory` 用于指定导出文件存储的目录的路径。

成功导出的文件内容为：

```
{
  "meta_data": {
    "chatcmpl": "chatcmpl-2e1aa823e2c94ebdad66450a0e6df088",
    "content_type": "application/json",
    "group_id": "enterprise-tier-5",
    "moonpalace_id": "13",
    "request_id": "c07c118e-4dae-11ef-b423-62db244b9277",
    "requested_at": "2024-07-29 21:30:43",
    "server_timing": "1033",
    "status": "200 OK",
    "user_id": "cn0psmmcp7fclnphkcpq"
  },
  "request": {
    "url": "https://api.moonshot.cn/v1/chat/completions",
    "header": "Accept: application/json\r\nAccept-Encoding: gzip\r\nConnection:",
    "body": {}
  },
  "response": {
    "status": "200 OK",
    "header": "Content-Encoding: gzip\r\nContent-Type: application/json; charset=utf-8",
    "body": {}
  },
  "category": "goodcase",
  "tags": [
    "code",
    "python"
  ]
}
```

我们推荐开发者使用 [Github Issues](#) 提交 Good Case 或 Bad Case，但如果你不想公开你的请求信息，你也可以通过企业微信、电子邮件等方式将 Case 投递给我们。

 Kimi K2.6 模型已正式发布，长程代码编写能力更强更稳。

 api-feedback@moonshot.cn
开放平台



>
此页面对您有帮助吗？

 是

 否

< [ModelScope MCP 配置](#)

[开发工作台调试](#) >

